

安全关键代码检查清单

强制要求：每次代码修改后必须执行

本检查清单必须在每次代码修改后执行，无论修改多小。一行代码的改动就可能引入致命缺陷。在修复本清单发现的任何问题后，必须重新执行完整清单——修复本身可能引入新问题。

不要跳过任何章节。不要假设某个章节不相关。

阶段1：编译与语法

- ☐ 代码编译无错误
- ☐ 代码编译无警告（将警告视为错误）
- ☐ 所有引用的头文件存在且路径正确
- ☐ 没有隐式函数声明
- ☐ 没有类型不匹配警告
- ☐ 没有未使用变量警告
- ☐ 所有函数原型与定义匹配

阶段2：逻辑正确性

逐行分析每个被修改的函数：

- ☐ 所有条件分支都处理了true和false两种情况
- ☐ switch语句都有default分支
- ☐ 循环终止条件正确（无差一错误）
- ☐ 循环变量没有在循环内被意外修改
- ☐ 所有函数调用的返回值都已检查
- ☐ 错误码沿调用链正确传播
- ☐ 指针参数在使用前已验证（NULL检查）

- [] 数组索引在访问前已做边界检查
- [] 整数运算不会溢出或下溢
- [] 不可能发生除以零（分母已验证）
- [] 类型转换安全（无截断、无符号问题）
- [] 枚举值覆盖了所有有效情况

阶段3：内存安全

- [] 每个 `malloc` / `platform_malloc` 都有对应的 `free`
- [] 每次分配都检查了NULL
- [] 任何代码路径上都不可能出现释放后使用
- [] 任何代码路径上都不可能出现重复释放
- [] 释放后指针已置为NULL
- [] 错误路径释放了错误发生前已分配的所有资源
- [] 任何代码路径上都没有内存泄漏（包括错误路径、提前返回和类异常跳转）
- [] 复制操作前已验证缓冲区大小
- [] `memcpy` / `memmove` 的大小不会超过目标缓冲区
- [] 没有使用 `strcpy`（使用 `strncpy` 或 `snprintf`）
- [] 没有使用 `sprintf`（使用 `snprintf`）
- [] 没有使用变长数组（VLA）
- [] 线程中的栈使用在配置的栈大小范围内
- [] 线程中大型局部变量已评估栈使用情况，栈空间紧张时使用堆分配

阶段4：线程安全与并发

- [] 共享变量已受保护（互斥锁、临界区或原子操作）
- [] 任何共享状态上都没有数据竞争
- [] 互斥锁的加锁/解锁始终配对（包括错误路径）
- [] 不可能发生死锁（一致的加锁顺序）
- [] 没有优先级反转风险（或已通过优先级继承缓解）
- [] ISR处理函数极简（仅设置标志/投递到队列）
- [] ISR上下文中没有阻塞调用
- [] ISR上下文中没有内存分配

- ☐ 回调已文档标注其运行的上下文
- ☐ 与ISR共享的变量使用了volatile限定符

阶段5：可重入性

- ☐ 从多个上下文调用的函数是可重入的
- ☐ 共享函数中没有隐藏的静态/全局可变状态
- ☐ 如果函数不可重入，则通过模块API序列化访问
- ☐ 中断安全函数不调用不可重入的函数

阶段6：栈溢出风险

- ☐ 没有递归（或递归深度严格有界且已证明安全）
- ☐ 线程函数中大型局部变量的栈使用已评估
- ☐ 调用链深度合理
- ☐ 没有使用 alloca()
- ☐ 没有使用VLA
- ☐ 栈大小配置为最深调用链的实际用量加足够余量

阶段7：资源生命周期

- ☐ 所有打开的文件/句柄在每条代码路径上都已关闭
- ☐ 所有获取的锁在每条代码路径上都已释放
- ☐ 所有启动的定时器在不再需要时已停止
- ☐ 所有创建的线程在关闭时已正确join/终止
- ☐ 所有注册的回调在清理时已注销
- ☐ 初始化和清理是对称的（清理时撤销初始化的所有操作）

阶段8：硬件交互（如适用）

- ☐ 硬件访问函数已验证到寄存器级别
- ☐ HAL调用的阻塞行为已验证
- ☐ 访问前已启用正确的外设时钟
- ☐ GPIO/引脚配置与硬件设计匹配

- ☐ 中断优先级遵循项目方案
- ☐ DMA配置与外设需求匹配
- ☐ 硬件操作之间的必要延时已就位
- ☐ 新代码路径不会导致看门狗饿死

阶段9：鲁棒性

- ☐ 所有外部输入已验证（范围、大小、格式）
- ☐ 所有等待操作都有超时机制
- ☐ 所有代码路径上都在喂看门狗
- ☐ 错误恢复不会使系统处于不一致状态
- ☐ 非关键故障有优雅降级机制
- ☐ 没有没有退出条件的无限循环

阶段10：代码质量

- ☐ 没有魔法数字（所有常量定义为宏）
- ☐ 没有死代码/未使用代码/被注释掉的代码
- ☐ 函数长度 ≤ 80 行
- ☐ 嵌套深度 ≤ 4 层
- ☐ 行宽 ≤ 80 列
- ☐ 函数参数 ≤ 5 个（超过则组合成结构体）
- ☐ 注释使用英文且与代码一致
- ☐ 修改的代码已更新注释
- ☐ 注释解释"为什么"，而非"是什么"
- ☐ 遵循项目命名约定
- ☐ 头文件包含路径匹配项目约定
- ☐ 头文件有头文件保护宏
- ☐ 头文件是自包含的（可独立编译）
- ☐ 所有未修改的指针参数使用 `const`
- ☐ 非公共函数/变量声明为 `static`
- ☐ 没有布尔参数——使用枚举提高清晰度
- ☐ 名称揭示意图（无无上下文的缩写）

阶段11：设计原则（SOLID与Clean Code）

- ☐ 每个函数只做一件事（SRP）
- ☐ 函数内的操作处于相同的抽象层级
- ☐ 查询类函数没有隐藏的副作用（CQS）
- ☐ 模块具有单一职责（一个变更理由）
- ☐ 新行为通过扩展添加，而非修改（OCP）
- ☐ 接口的所有实现遵守相同的契约（LSP）
- ☐ 使用者只依赖其使用的接口（ISP）
- ☐ 高层模块依赖抽象，而非具体的底层模块（DIP）
- ☐ 没有重复的代码块（DRY）
- ☐ 适当时用表驱动设计替代复杂的if-else/switch
- ☐ 断言用于内部契约
- ☐ 运行时检查用于外部输入
- ☐ 错误码被传播，而非静默吞掉

阶段12：影响分析

在确认修改的代码本身正确后，验证修改不会破坏其他部分：

- ☐ 被修改函数的所有调用者仍正常工作
- ☐ 所有被调用者仍收到有效参数
- ☐ 结构体布局变更已反映到使用该结构体的所有地方
- ☐ 枚举/宏值变更已反映到所有switch语句和比较中
- ☐ 接口变更已反映到所有实现中
- ☐ 共享头文件变更与所有包含者兼容
- ☐ 初始化顺序仍然正确
- ☐ 关闭/清理序列仍然正确

执行协议

1. 按阶段1-12顺序逐一执行
2. 对每个检查项主动验证——不要假设

- 3. 如果发现任何问题： a. 修复问题 b. 从阶段1开始重新执行完整清单 c. 重复直到所有阶段都干净通过
- 4. 记录发现的任何假设或限制
- 5. 向用户报告发现，包括： - 发现并修复的问题 - 需要人工审查的潜在风险 - 对硬件或平台行为做出的假设

严重性分级

报告问题时：

严重性	描述	操作
致命	可能导致系统故障、数据损坏或崩溃	必须在其他工作之前修复
高	内存泄漏、竞态条件、资源泄漏	必须在本次会话中修复
中	代码质量违规、缺少校验	应立即修复
低	风格问题、小优化	时间允许时修复

对于安全关键代码，致命和高严重性问题是不可商量的 ——在代码修改被视为完成之前必须解决。

获取更多

- **B站搜索「兆鸣嵌入式」** 观看完整教程（详细讲解+代码实战）
- 公众号搜索「兆鸣嵌入式」回复「交流」加技术交流群
- 抖音搜索「兆鸣嵌入式」看短视频精华版